Week 10 Lab – Autonomous Valet Parking

This lab will guide you through creating an autonomous valet parking map. **Your goal will be to use real-world self-driving car data to build a "map" of parking spaces.** You will create a full package that can generate this map.

Lab 10: Autonomous Valet Parking

This exercise will provide you with a bare package that contains a simple car URDF, as well as a script that transforms a pose message to a TF frame. Your task will be to take these parts, along with a rosbag of real-world data, and create 3 launch files that run the bags, urdf and an octomap server.

1. **Get the Package:** Download the "lab10" package from SurreyLearn and extract it into a new workspace, under the src directory. Inspect the contents of the package, then compile and source your workspace, adding any relevant lines to your ~/.bashrc file.
2. **Play the rosbag**: Using the command

```
$ rosbag play /vol/vssp/signsrc/baxter_ws/carpark.bag
```

   play the rosbag that contains the self-driving car data. Open RVIZ and inspect the contents of the rosbag. You should be able to visualise 8 images, IMU, LiDAR points and Vehicle pose. Figure out what the topic names for each of these things are. Remember that there is no TF tree at the moment, so you will need to switch to each sensors coordinate frame to visualise them.

3. **Write a Launch file player:** Once you are happy with the contents of the rosbag, write a launch file that plays it. Make a new file called "reconstruct.launch" in the "launch" folder of your package. To play the rosbag, add the following node:

```
<node name="player" pkg="rosbag" type="play" output="screen"
                                 args="--clock -r $(arg bagspeed) $(arg bagfile)"/>
```

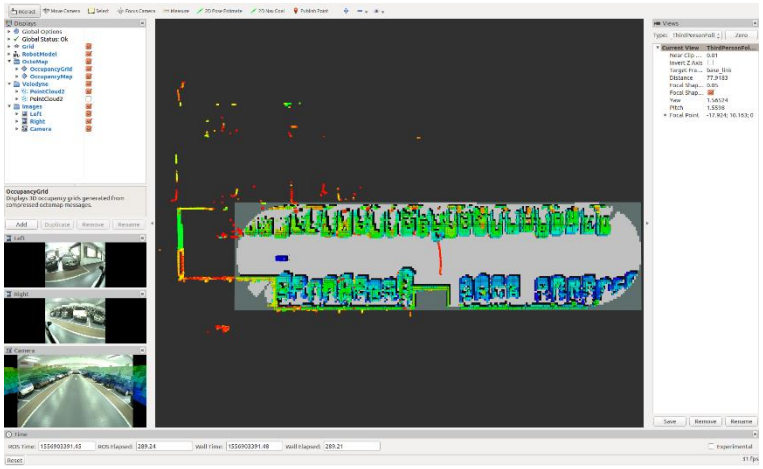
   remember to add *"bagspeed"* and *"bagfile"* as arguments to your launch file. These should have default values of "1" for bagspeed and the path from step 1 for bagfile. The bagspeed parameter will help during debugging. Play this launch file and ensure that the rosbag plays and the same data is visible in RVIZ.

4. **Write a URDF launch file:** Make a new file called urdf.launch in the launch folder of your package. This launch file will be in charge of loading the Car's URDF model. To do so, add the following code to the launch file:

```
<param name="robot_description" command="$(find xacro)/xacro $(arg model)" />
<param name="use_gui" value="$(arg gui)"/>
```

   and remember to set the "gui" and "model" parameters. The "gui" parameter should default to false, while the "model" parameters should default to the URDF file in your package, use the $(find <package-name>) interface! As usual, this file should start the "**joint_state_publisher**" and "**robot_state_publisher**" nodes. Finally, this launch file should start the "**pose_to_tf.py**" node. Once this launch file is working, add it as an include to "reconstruct.launch".

5. **Visualise URDF and Bag:** To verify that your URDF launch file is working, run "reconstruct.launch" and open RVIZ using the config file "rviz/car.rviz" in your package. You should be able to visualise the car driving around, all sensors and even the absolute pose of the vehicle relative to map. Your RVIZ window should look like this:



(without the OctoMap, which we will add below).

6. **Add RVIZ to Launch File:** In "reconstruct.launch", add an "rviz" node that uses the config file in your package:

```
<node name="rviz" pkg="rviz" type="rviz" args="-d $(find
<package_name>)/rviz/car.rviz" required="true" />
```

remember to change **<package_name>** to your package's name.

7. **Create an OctoMap:** Once you have a working version of RVIZ, URDF and the rosbag, we can start to create the OctoMap that will provide the floorplan. Create a new launch file called "octomap.launch". To run the OctoMap, use the following code:

```
<node pkg="octomap_server"  type="octomap_server_node" name="octomap_server">
      <remap from="cloud_in"                    to="$(arg pointcloud)"/>
      <param name="frame_id"                  value="<map_frame_id>" />
      <param name="base_frame_id"             value="<car_frame_id>"/>
      <param name="sensor_model/max_range"    value="10" />
</node>
```

Make sure you set the "pointcloud" parameter in your launch file. **Note:** Look at the documentation for the octomap_sever (http://wiki.ros.org/octomap_server) and try to figure out what the **<map_frame_id>** and **<car_frame_id>** should be. These parameters will make a massive impact to the performance of the system, so make sure you understand what they mean before proceeding.

8. **Visualise the OctoMap:** Add "octomap.launch" as an include in "reconstruct.launch". Launch "reconstruct.launch". The provided RVIZ config file should already include the OctoMap and its floorplan. Make sure you can see the OctoMap being built as the car drives around.

9. **Tune OctoMap:** You may notice that the octomap cannot keep up with the rosbag's speed. This is because the resolution is too high, making the construction of the octree slow. Using

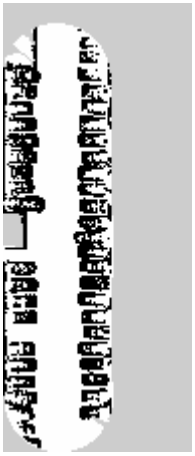
the documentation, figure out what parameter to change to make each “cube” in the Octree larger.

10. **Tune Floorplan:** You might notice that the floorplan is not particularly clean. Can you figure out why this is? Think about how the OctoMap is creating the floorplan by projecting all voxels to the ground plane. Once you have figured it out, use the following code:

```
    <node pkg="nodelet" type="nodelet" name="pcl_manager" args="manager" />
    <node pkg="nodelet" type="nodelet" name="passthrough" args="load
pcl/PassThrough pcl_manager" output="screen">
        <remap from="~input" to="$(arg pointcloud)" />
        <remap from="~output" to="$(arg pointcloud)_filtered" />
        <rosparam>
          filter_field_name: <x,y or z>
          filter_limit_min: -100
          filter_limit_max: 100
          filter_limit_negative: False
        </rosparam>
    </node>
```

to filter the pointcloud in the correct way. Make sure you rename the input topic to the OctoMap! If you are struggling with this, ask the demonstrators for help!

11. **Fine Tuning:** If required, fine tune the parameters of the system so that your floorplan looks like this:



notice how each bay is easily identifiable and there is no noise from the floor or ceiling.

12. **Finished:** Once you get the map above, the lab is finished. Well done! You have finished the last lab in the course!

13. **Extra challenge:** If you want to tackle an extra challenge, can you identify the empty bay?
    Hints:
    a) The bays are “fixed” in place, you can figure out where the edges of them are!
    b) The cars appear as “occupied” space in both the Octree and the 2D Map!